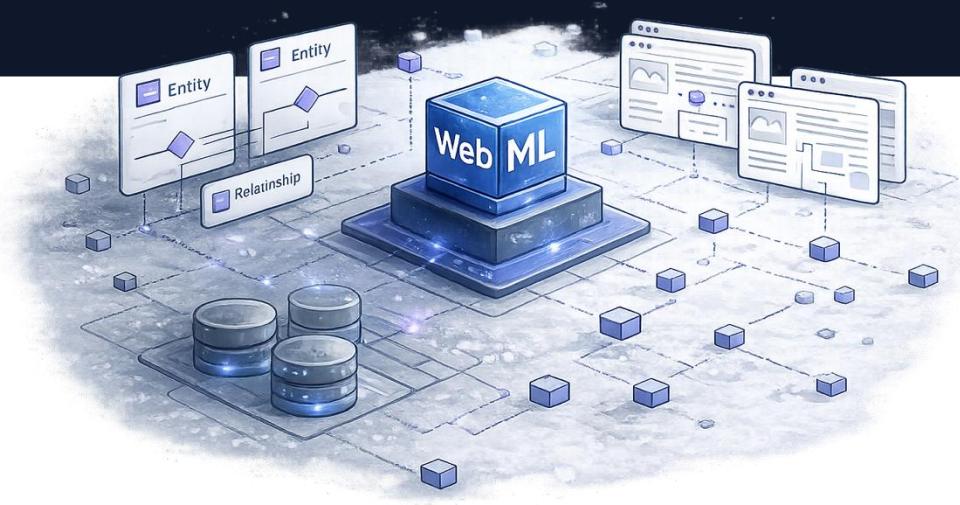




Progettazione di applicazioni web full stack

Progettazione model-driven dei contenuti ipertestuali



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia

Progettazione dell'ipertesto - Obiettivi



- Modellazione ad alto livello del front-end di un'applicazione Web dinamica e delle interazioni con la logica e i dati del back-end
- Utilizzo di una notazione visuale e concettuale semplice ma formale
- Progettazione di (*template*) di pagine dinamiche e di interrogazioni per l'accesso e la manipolazione dei dati



1. In che modo l'utente deve fruire del contenuto pubblicato tramite il sito?
2. Quali sono le pagine nell'ipertesto, tramite cui l'utente può accedere ai contenuti?
3. Quale informazione deve essere pubblicata in ogni pagina?
4. In che modo i nodi dell'ipertesto (per es., le pagine) sono collegati tra loro?

Fase di progettazione dell'ipertesto



- **IN**: schema dei dati, mappa del sito, requisiti funzionali, requisiti utente
 - **OUT**: schema di ipertesto WebML
-
- Due passi di progettazione:
 - 1) *Progetto coarse* (preliminare)
 - 2) *Progetto dettagliato*
 - Fondamentale il concetto di «*site view*»: «*vista sul sito*» per ciascun gruppo utente



- Per ogni «site view»:
 - 1) *Individuazione delle aree*
 - revisione dei requisiti funzionali e della mappa del sito (suddivisione delle site view in sotto-parti coese)
 - 2) *Definizione della visibilità delle aree*
 - area di default, area landmark, area interna (raggiungibile attraverso link espliciti)
 - 3) *Specifica dei contenuti tramite primitive ad-hoc*

Esempi di «aree»: l'insieme di pagine pensate per la gestione dei libri nel catalogo, per la gestione degli autori, per la gestione delle categorie

- **Pubblica:** accesso consentito ad ogni utente
- **Privata:** accesso consentito tramite autenticazione e autorizzazione

Primitive del progetto coarse (I)



Core (CoreEntity, Component1,... ComponentN)

- Una primitiva **Core** denota la pubblicazione di contenuti relativi a entità core e sue componenti

Primitive del progetto coarse (II)



Access (CoreEntity, AccessEntity1,... AccessEntityN)

- Una primitiva **Access** denota l'accesso a una o più istanze dell'entità core attraverso entità di categorizzazione o specializzazione (sottoschemi di accesso)



Interconnection (Entity1, Role1,... RoleN)

- Una primitiva **Interconnection** denota la navigazione da istanze di **Entity1** a istanze di altre entità core connesse da ruoli di relazione **Role1..RoleN**

Primitive del progetto coarse (IV)



Personalization (Entity1, Role1,... RoleN)

- Una primitiva **Personalization** denota la pubblicazione di contenuti filtrati a partire dal profilo di un utente autenticato

Primitive del progetto coarse (IV)



```
Create (Entity)  
Create&Connect(Entity,Role1,... RoleN)  
Modify (Entity)  
Delete (Entity)
```

- Le primitive **Create**, **Modify**, **Delete** denotano le omonime operazioni CUD sulle istanze di entità del modello dei dati
- La primitiva Create è spesso utilizzata insieme alla primitiva **Connect** (**Create&Connect**) a formare pattern di creazione

Progetto coarse – Esempio (I)



Area libri

Core (Book)

Create&Connect (Book, Bok2Category, Book2Author)

Modify (Book)

Delete (Book)



1) *Individuazione delle pagine*

- Suddivisione di aree in pagine
- Ogni pagina include una porzione di contenuto e di funzioni dell'area che la include

2) *Visibilità di pagina*

- Home page
- Pagina di Default
- Pagina Landmark
- Pagina Interna

3) *Specifica di Pagina*

- Utilizza sotto-schemi di ipertesto e design pattern tipici
- Un sottoschema di ipertesto per ogni sottoschema dei dati

Progetto dettagliato - Esempio



Catalogo libri

Core (Book)

Delete (Book)

Crea libro

Create&Connect (Book, Bok2Category, Book2Author)

Modifica libro

Modify (Book)



- Una pagina è un contenitore di una o più unità di contenuto mostrate all'utente *contemporaneamente*

Mark per denotare la **home page** (pagina principale del sito o della site view, su cui si atterra al primo accesso o dopo il login)

HomePage

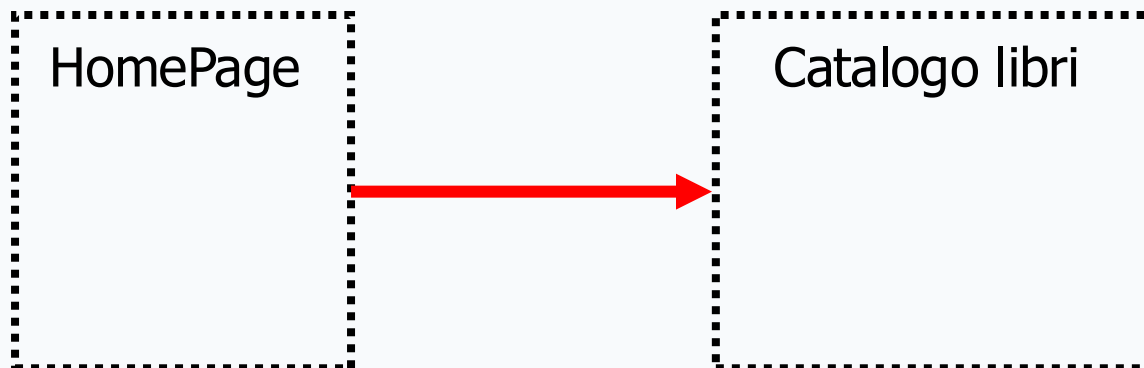


Indice dei libri

Link non contestuali



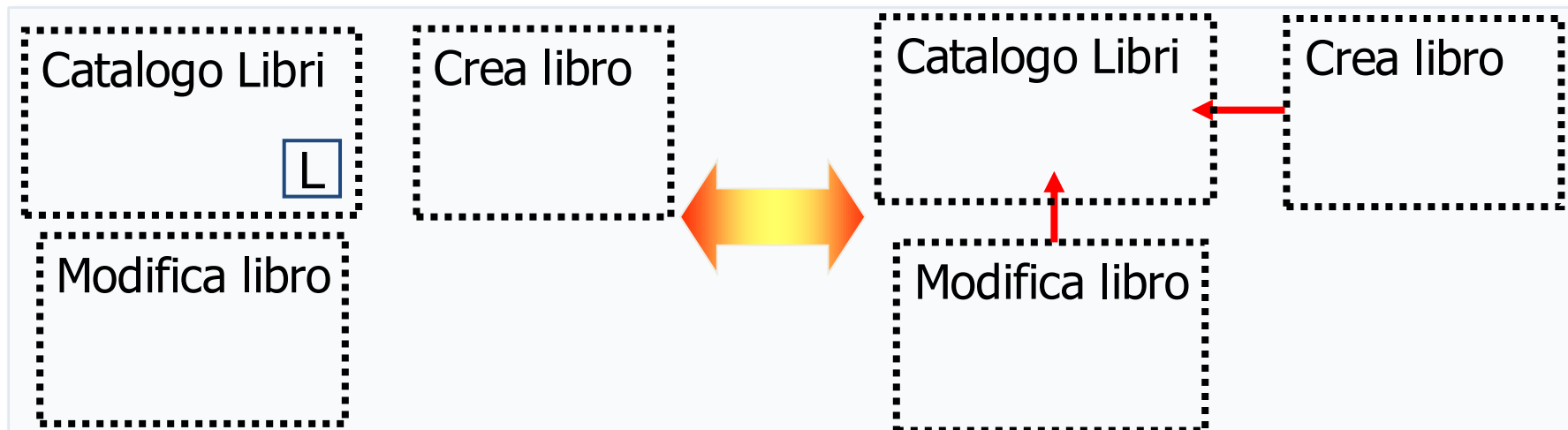
- I link tradizionali che permettono di navigare da una pagina all'altra
 - *Non trasportano nessuna informazione di contesto*



Pagine «landmark»



- Pagine globalmente visibili → l'utente può saltare ad esse da ogni altra pagina della site view
- Equivalenti a *link non contestuali* definiti da ogni altra pagina della site view verso la pagina landmark



Aree e pagine «landmark»



- Il raggruppamento di pagine dentro aree «*landmark*» e l'identificazione di pagine «*landmark*» in fase di progettazione determina la definizione della cosiddetta «*navbar*» del sito web



- Primitive del modello ipertestuale
 - *Unità di visualizzazione del contenuto (o **View Components**)*
 - *Link contestuali*
 - *Unità di creazione, modifica, cancellazione istanze (**Operation**)*
 - *Pagine alternative (**XOR**)*
 - *Primitive di login/logout*
 - *Parametri di contesto (per modellare lo stato)*
 - *Primitive per modularità e riuso*



- Una *unità di visualizzazione del contenuto* in WebML è l'elemento base per la pubblicazione di informazioni
- Corrisponde ad una «vista» definita su un contenitore di oggetti, ad es.:
 - Le istanze di un' *entità sorgente*
 - Le istanze di una entità che soddisfano una condizione di selezione chiamata *selettore*



Unità di contenuto di base



Details



Libro
[ID==1]

SimpleList



Libro
[pubY==2026]

MultipleDetails



Libro
[pubY==2026]

Form



Libro

Scroller



Libro
[ID==1]

List



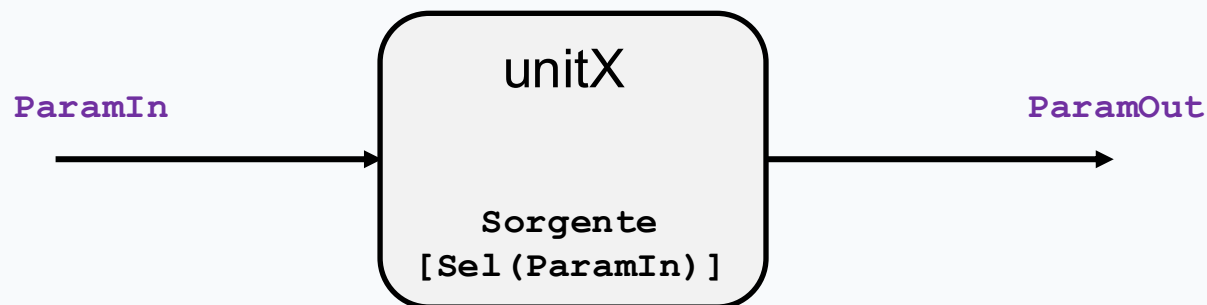
Libro
[Autore2Libro]

Hierarchy



Libro
[PubY==2026]
Libro2Autore

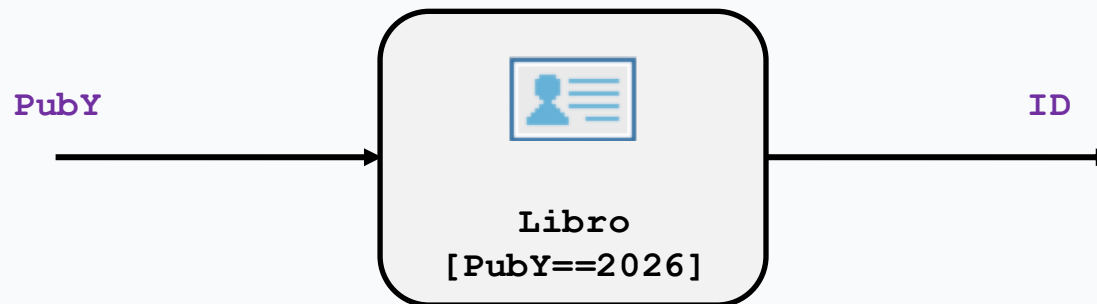
Input e output delle unità di contenuto



- Ogni unità di contenuto può avere *parametri di input e output*
- I parametri in input sono necessari per calcolare l'unità
 - *Parametri richiesti dal selettore della unit*
- I parametri in output possono essere utilizzati per la computazione di *una o più unità che dipendono da quella corrente*



- Diverse modalità di selezione possono essere utilizzate all'interno di una *view component*
 - **[ID==?]** – seleziona le istanze dell'entità sorgente i cui ID sono quelli passati come input
 - **[condition]** – seleziona le istanze dell'entità sorgente che rispettano la condizione specificata (e.g., **[pubY==2026]**)
 - **[relation]** – seleziona le istanze dell'entità sorgente che sono connesse dalla relazione «*relation*» agli ID passati come input (e.g., **[Autore2Libro]** mostra le istanze di libro collegate agli ID degli autori passati come input)



Pubblica informazione relative ad una *singola istanza*

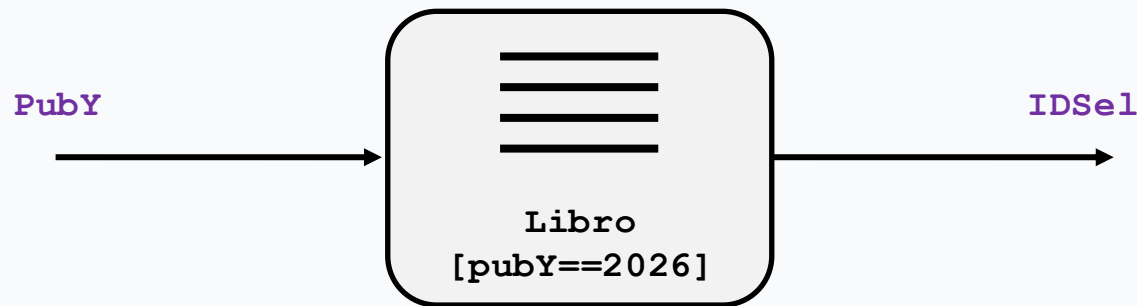
Contenitore: una entità, (opzionalmente) un selettore

Parametri in input: quelli richiesti per la compilazione del selettore

Parametri in output:

- L'**ID** dell'oggetto pubblicato, con ogni suo attributo

SimpleList unit



Pubblica una lista di elementi (*insieme di oggetti*); ogni element è «*cliccabile*»

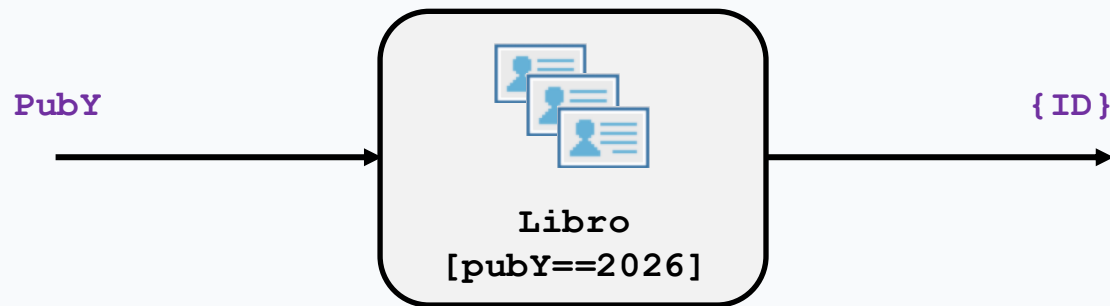
Contenitore: una entità, (opzionalmente) un selettore

Parametri di input: quelli richiesti per la computazione del selettore

Parametri in output:

- L'**ID** dell'oggetto selezionato dall'utente

MultipleDetails unit



Pubblica un insieme di istanze di una entità (*insieme di oggetti*)

Contenitore: una entità, (opzionalmente) un selettore

Parametri di input: quelli richiesti per la computazione del selettore

Parametri di output:

- L'insieme di **ID** degli oggetti pubblicati



(opzionale) può essere indicata un'entità, a significare che la form conterrà tutti i campi necessari a valorizzare gli attributi di quell'entità

{parametri}

Per l'*immissione di dati* da parte dell'utente: campi per introdurre valori, campi di selezione per selezionare tra più valori

I risultati vengono passati come parametri ad altre unità o al back-end

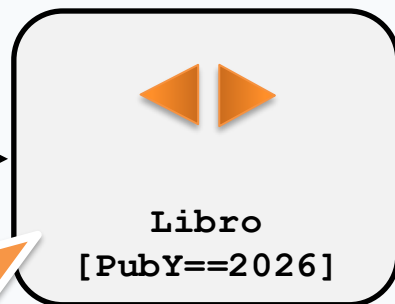
Contenitore: i campi della form

Parametri in output: i valori dei campi inseriti

Scroller unit

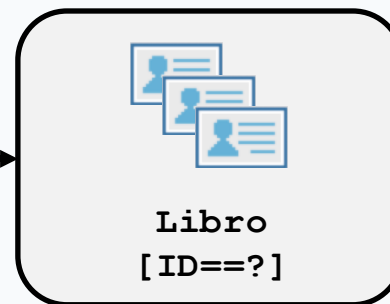


PubY



[Block factor]

{ ID }



{ ID }

La Scroller Unit va sempre usata in combinazione con un'unità pensata per visualizzare un elemento

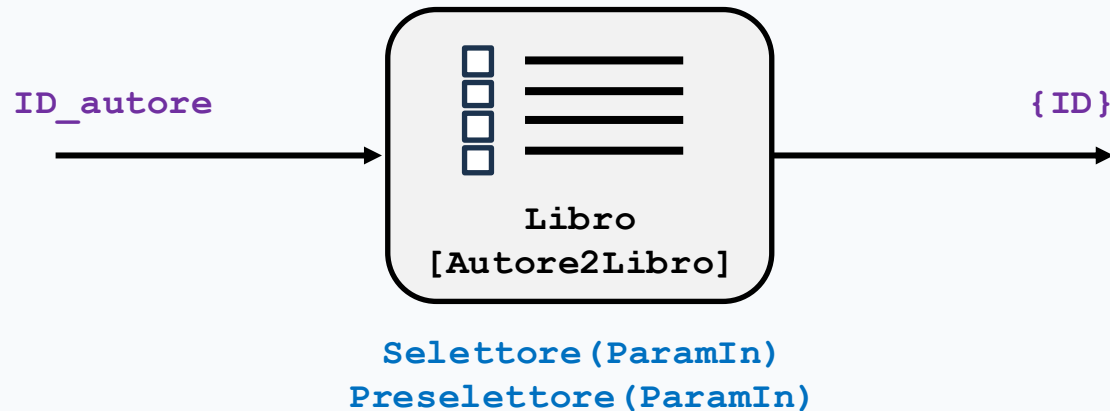
Permette di definire la *paginazione* in un insieme di oggetti: visualizza link al primo, al precedente, al prossimo, all'ultimo oggetto nell'insieme

Contenitore: una entità, (opzionalmente) un selettore

Parametri in input: quelli richiesti per la computazione dei selettori

Parametri in output: l'insieme di ID del blocco di oggetti corrente

Block factor: numero di oggetti visualizzati in blocco in un passo di navigazione

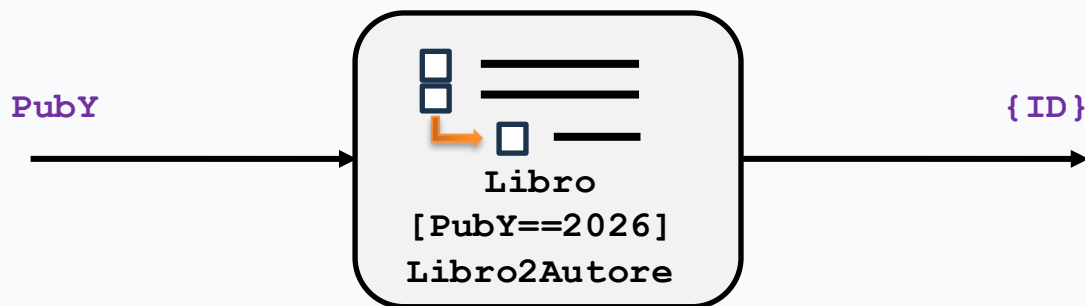


Pubblica una lista di elementi (*insieme di oggetti*) tra cui l'utente seleziona uno o più elementi (tramite *checkbox*)

Contenitore: una entità, (opzionalmente) selettore e pre-selettore

Parametri in input: quelli richiesti per la computazione dei selettori

Parametri in output: Gli **ID** degli oggetti marcati dall'utente



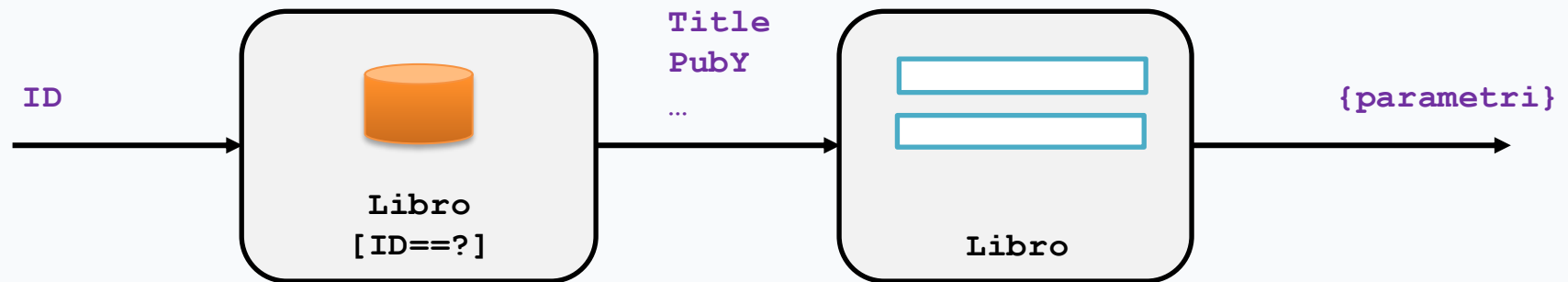
Pubblica una lista di oggetti (*appartenenti a più entità*) organizzati *gerarchicamente* in base a relazioni definite tra le entità

Contenitore: un insieme di entità e le relazioni che le associano, (opzionalmente) selettori ad ogni livello

Parametri in input: quelli richiesti dalla computazione dei selettori

Parametri in output: Gli ID degli oggetti selezionati dall'utente

Selector Unit



- Un'unità aggiuntiva, pensata non per visualizzare contenuti, ma per richiedere contenuti dal back-end, allo scopo di passarli ad altri *view components*
- Usata spesso in combinazione con la **Form Unit** per rappresentare il pre-caricamento di informazioni nei campi della form (voci di menu a tendina, form per la modifica di contenuti)

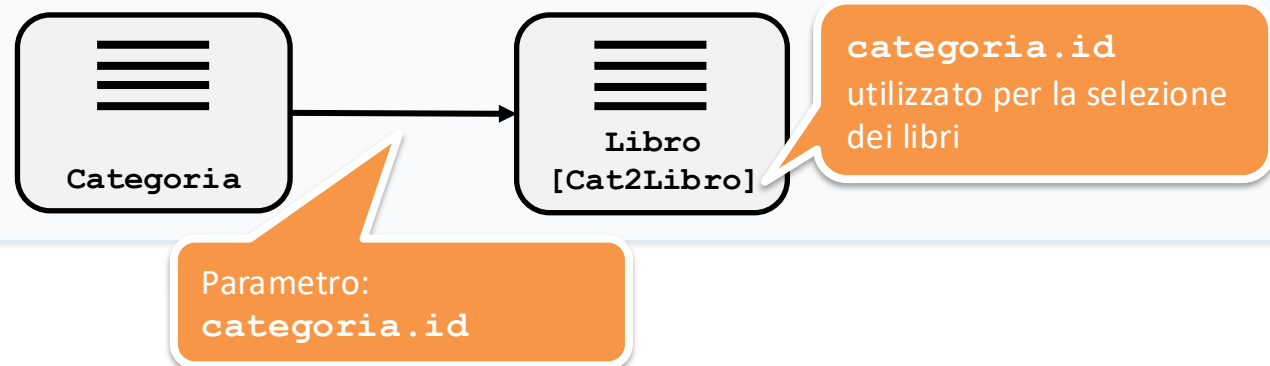


- Connessioni orientate tra due unità (*sorgente* e *destinazione*):
 - Permettono all'utente di navigare tra nodi diversi dell'ipertesto
 - Trasportano informazione di contesto
 - Possono attivare una computazione
 - La loro implementazione può corrispondere ad ancore o bottoni «*submit*»

Link contestuali (II)



- Il contesto è trasportato dai link attraverso l'uso di parametri definiti sui link
 - Un parametro sui link ha un **Nome**
 - Il contenuto del parametro è tipicamente utilizzato nel selettore dell'unità destinazione
 - *Laddove possibile, i parametri sui link possono essere lasciati impliciti* (i diagrammi acquistano maggior chiarezza)





- I link contestuali automatici passano il contesto dalla sorgente alla destinazione immediatamente dopo il rendering della sorgente, senza bisogno di alcun intervento da parte dell'utente



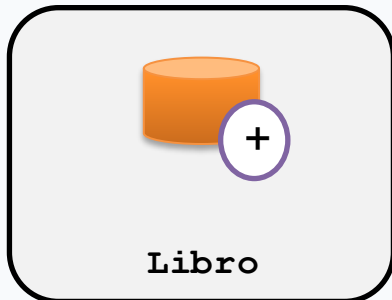


- Una *operazione* in WebML è l'elemento base per modellare una operazione di creazione, cancellazione o modifica di istanze delle entità
- Input da uno o più link entranti (almeno uno, che avvia l'operazione, definito *normal link*, gli altri sono *link di trasporto*)
- Due tipi di link in output:
 - **OK** (*verde*) se l'operazione è avvenuta con successo
 - **KO** (*rosso*) altrimenti

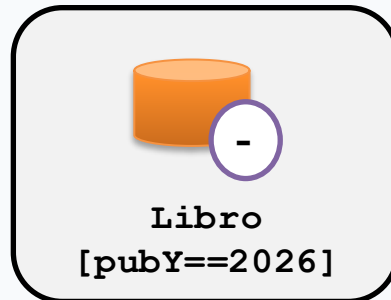
Operazioni di base



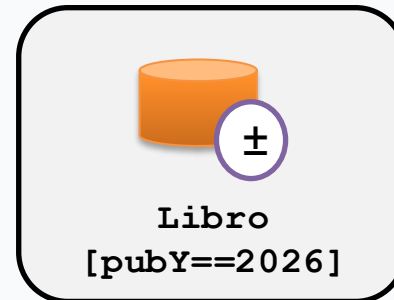
Create



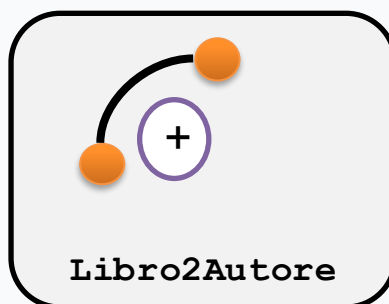
Delete



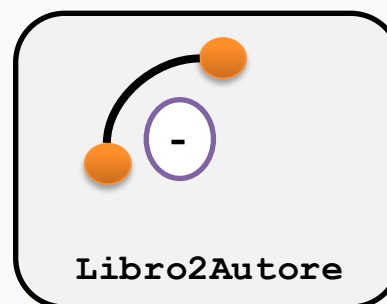
Update



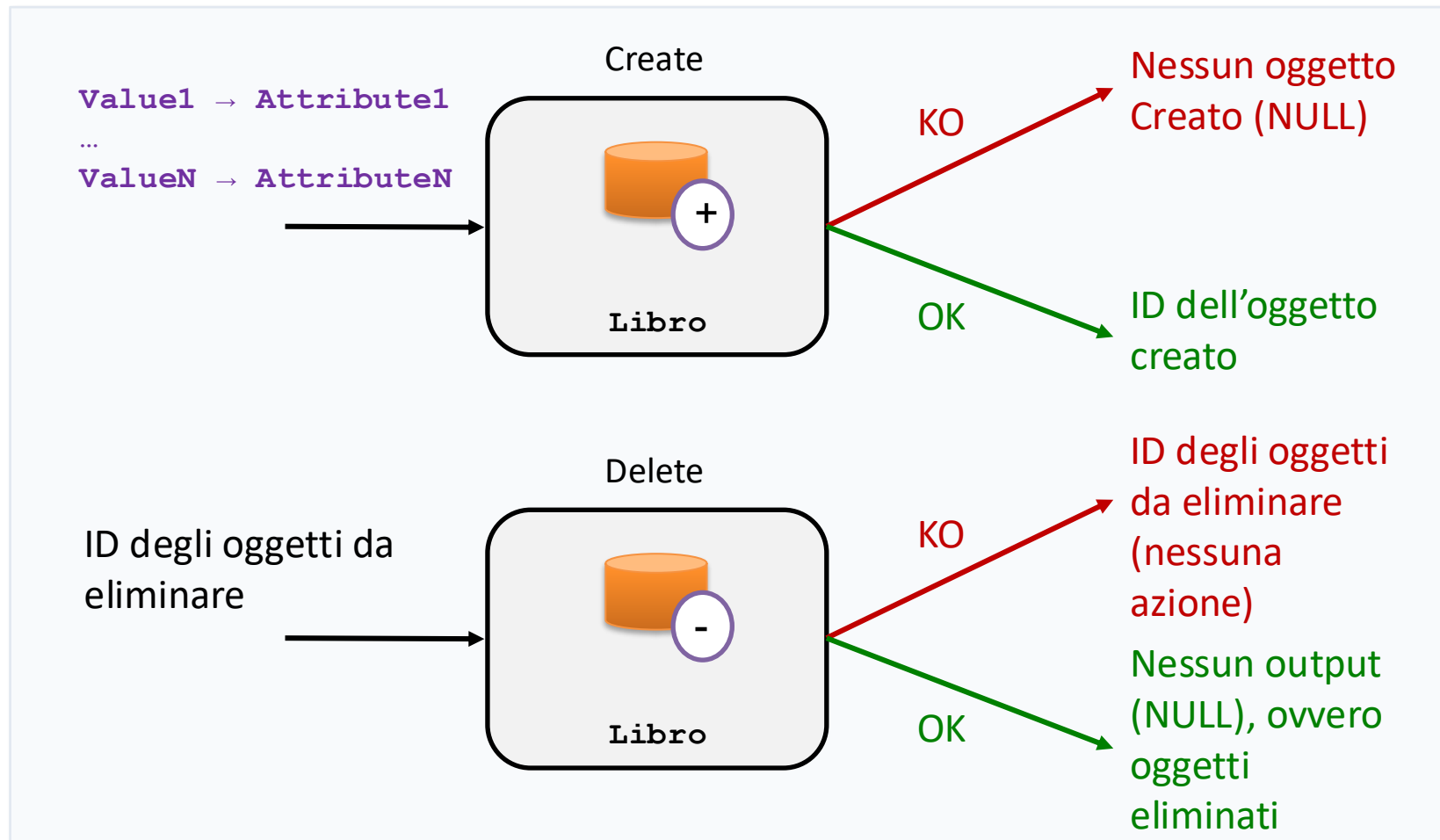
Connect



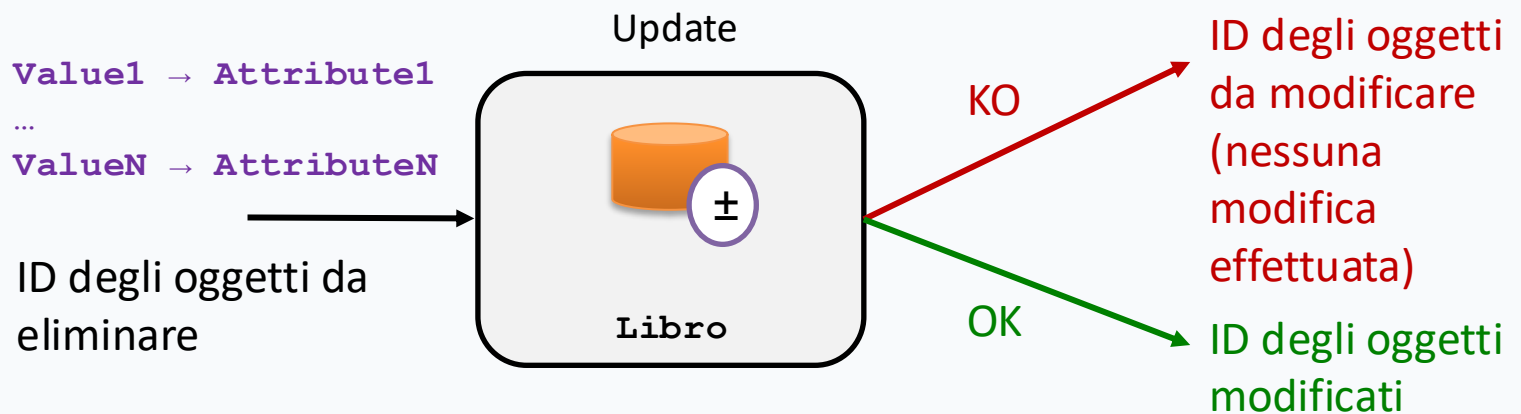
Disconnect



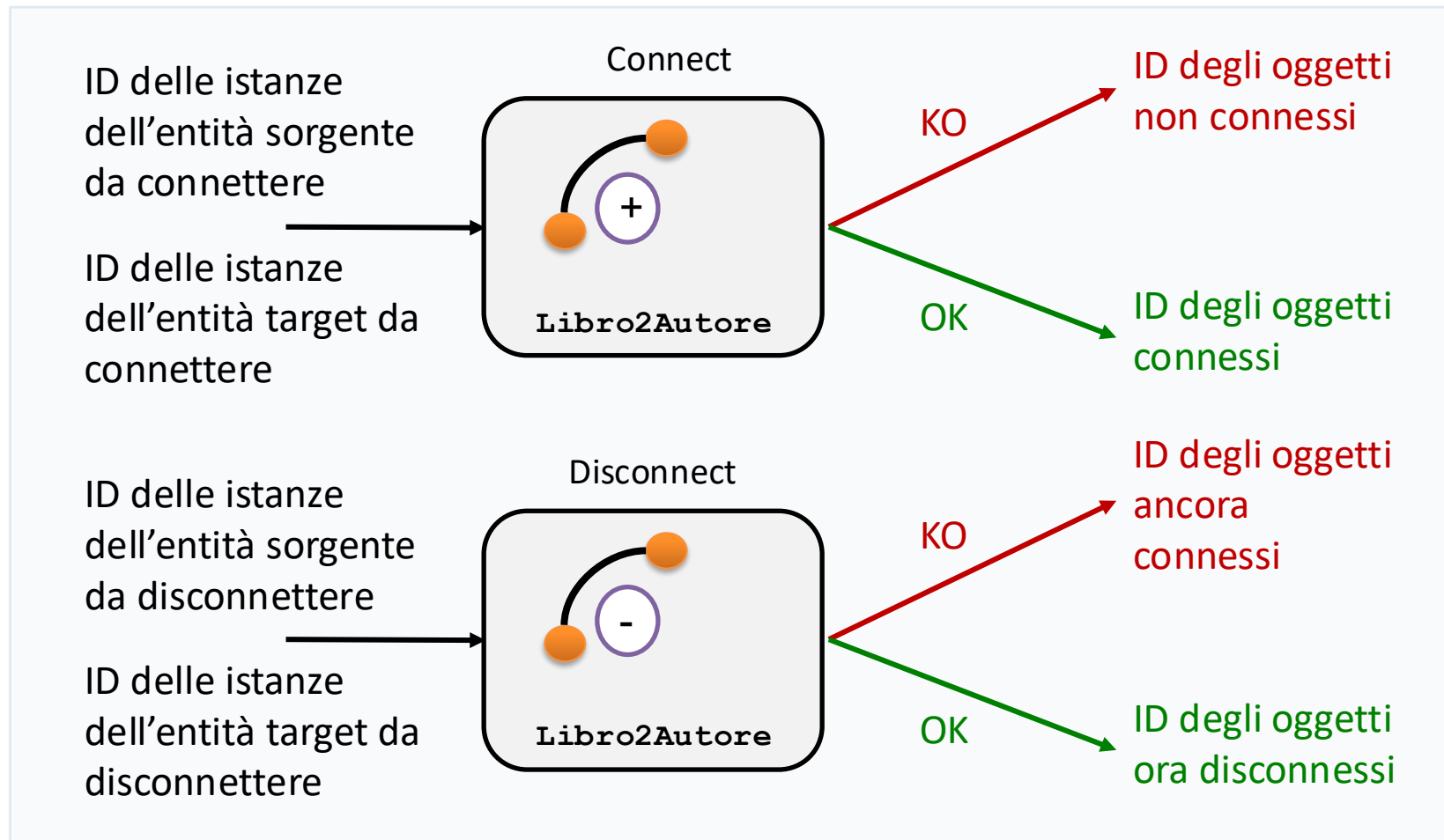
Operazioni di base – Flusso di I/O (I)



Operazioni di base – Flusso di I/O (II)

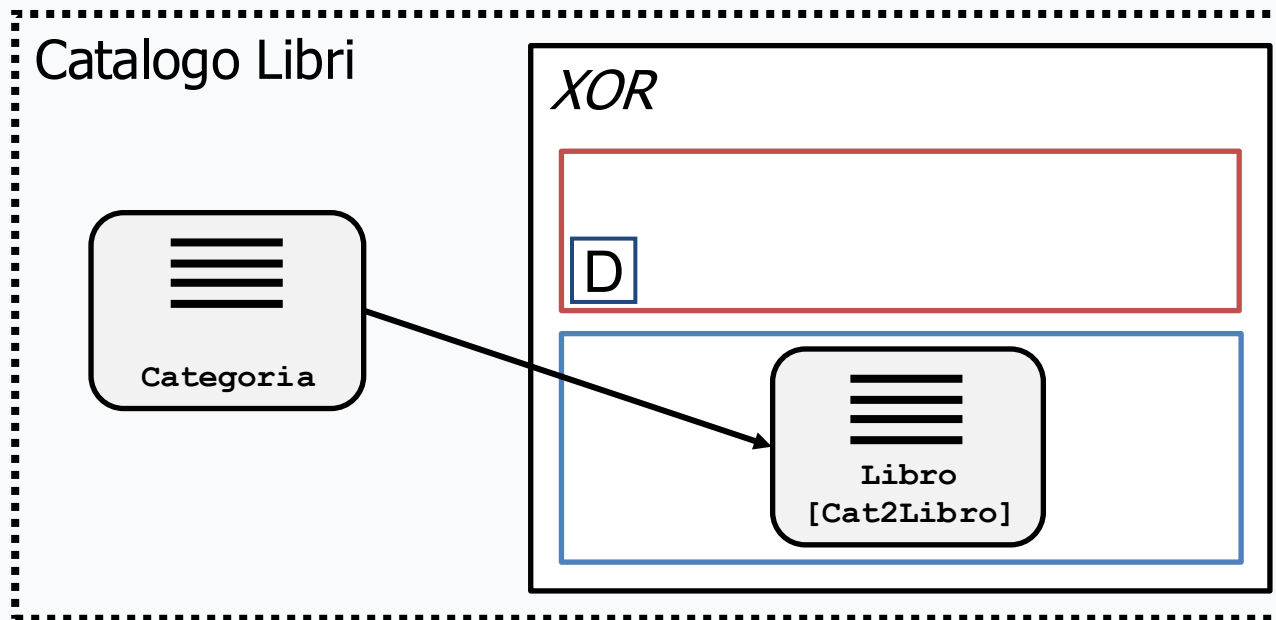


Operazioni di base – Flusso di I/O (III)





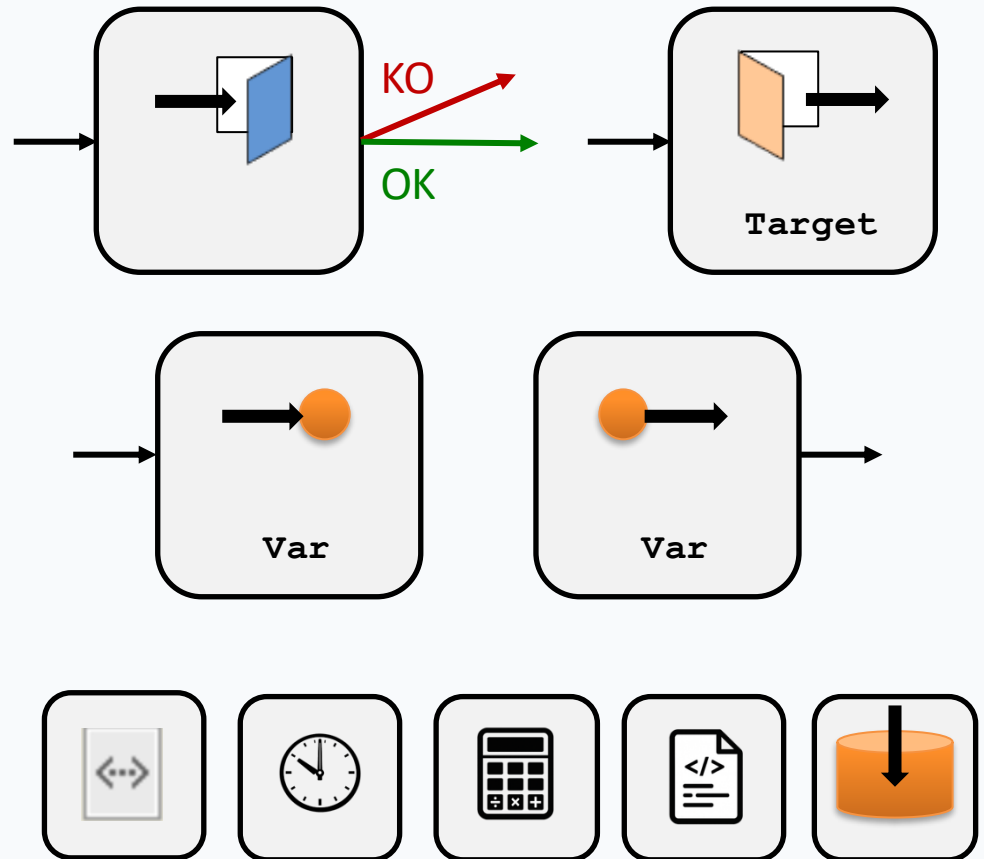
- Notazione utilizzata per rappresentare contenuti alternativi all'interno di una pagina web dinamica



Altri costrutti di modellazione (I)



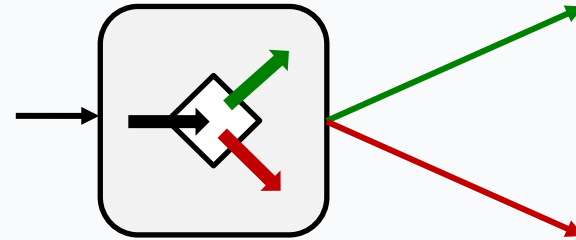
- Login/logout unit
- Set/get unit (per gestione variabili di stato)
- HTML statico, ora corrente, Math, script ad-hoc, query ad-hoc



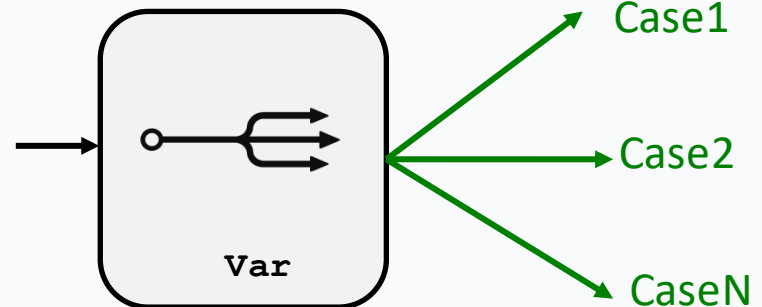
Altri costrutti di modellazione (II)



- IsNotNullUnit



- Switch



- Modularità e riuso



Progettazione di applicazioni web full stack

Progettazione model-driven dei contenuti ipertestuali



```
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

#selection at the end -add
obj.select= 1
obj.select=1
context.scene.objects.active
("Selected" + str(modifier))
mirror_ob.select = 0
bpy.context.selected_objects
data.objects[one.name].select
print("please select exactly")

-- OPERATOR CLASSES -----

types.Operator):
X_mirror = mirror_x"
```

Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia